Day 44 — Developer Handbook

1. Purpose

This handbook describes how to set up, test, integrate with, and troubleshoot the Zecpath ATS API and HR Interview AI Python components.

2. Repository Components

Location	Responsibility
`ats_engine/api.py`	FastAPI ATS endpoints
`interview_ai/interview.py`	Interview state and role-based questions
`interview_ai/follow_up_engine.py`	Adaptive follow-up decisions
`scoring/hr_interview_scoring_engine.py`	Interview answer and interview-level scoring
`scoring/communication_scoring_engine.py`	Communication evaluation
`scoring/confidence_sentiment_engine.py`	Confidence/sentiment indicators
`scoring/interview_summary_generator.py`	Recruiter-facing summary reports
`data/hr_screening_dataset.json`	Interview question dataset
`data/hr_interview_scoring_configuration.json`	Interview score weights and thresholds
`tests/`	Regression and component tests

3. Environment Setup

From the repository root in PowerShell:

```
.\venv\Scripts\Activate.ps1
python --version
python -m pip install -r requirements.txt
```

If PowerShell blocks activation, use an approved environment policy or invoke the environment's Python executable directly. Avoid changing machine-wide execution policy just to run the project.

4. Run the ATS API

Start the FastAPI application locally:

```
python -m uvicorn ats_engine.api:app --reload
```

Open:

- Interactive documentation: `http://127.0.0.1:8000/docs`
- OpenAPI schema: `http://127.0.0.1:8000/openapi.json`
- Health check: `http://127.0.0.1:8000/api/v1/health`

The server command assumes Uvicorn is installed in the active environment.

5. Call the API

Example PowerShell health check:

```
Invoke-RestMethod -Method Get -Uri "http://127.0.0.1:8000/api/v1/health"
```

Example JSON scoring request:

```
$body = @{
    job_role = "Data Scientist"
    scores = @{
        skill_match = 90
        experience_relevance = 80
        education_alignment = 75
        semantic_similarity = 85
    }
} | ConvertTo-Json -Depth 5

Invoke-RestMethod -Method Post `
    -Uri "http://127.0.0.1:8000/api/v1/scoring" `
    -ContentType "application/json" `
    -Body $body
```

For resume upload, use multipart form data with the required `file` field. Consult `/docs` for the exact schema and response details.

6. Use the Interview Engine from Python

The interview engine is a Python component, not a confirmed REST endpoint. A minimal initialization example is:

```
from interview_ai.interview import InterviewAI

engine = InterviewAI(
    role="Data Analyst",
    experience_level="fresher",
)
```

```
questions = engine.generate_questions()
print(questions)
```

Use only roles present in `data/hr_screening_dataset.json`. Unsupported roles or experience levels raise `ValueError`.

The follow-up engine can analyze a response:

```
from interview_ai.follow_up_engine import DynamicFollowUpEngine

follow_up = DynamicFollowUpEngine()
decision = follow_up.analyze_response(
    "I developed a dashboard using Python and SQL.",
    question_id="Q001",
)
print(decision)
```

The follow-up engine limits follow-ups to two and tracks its state. Create a separate instance per interview session to avoid mixing state across candidates.

7. Scoring Logic

The configured interview scoring weights are:

- Answer relevance: 35%
- Communication: 25%
- Confidence: 25%
- Consistency: 15%

Scores are normalized to 0?100. Configuration changes should be accompanied by tests that validate boundaries, component scores, and overall score behavior.

A first-answer consistency score defaults to 100 when no previous answer exists. This is a computational default, not evidence of actual consistency.

8. Testing

Run focused tests:

```
python -m pytest tests/test_ats_api.py -q --tb=short
python -m pytest tests/test_day33_interview_engine.py
tests/test_day37_hr_interview_scoring_engine.py -q --tb=short
python -m pytest tests/test_day39_interview_summary_generator.py
tests/test_day40_hr_interview_simulation.py -q --tb=short
```

Run the complete regression suite before committing:

```
python -m pytest --tb=short -q
```

Review `git diff --check` and `git status --short --branch` before staging.

9. Troubleshooting

API fails to start

- Confirm the virtual environment is active.
- Check `python -m pip show fastapi uvicorn`.
- Read the traceback and verify the module path.
- Start from the repository root so package imports resolve.

HTTP 422

- Check the JSON field names and required fields.
- Ensure score values are between 0 and 100.
- Ensure `resume_text` is a non-empty string.
- Use `/docs` to inspect the current OpenAPI contract.

HTTP 404 for job status

- Confirm the job ID came from a successful upload.
- The current job store is in memory and does not survive process restarts.

Unsupported interview role

- Check the role spelling against `data/hr_screening_dataset.json`.
- Use `fresher` or `experienced` as the experience level.

No follow-up generated

- The answer may be considered sufficient.
- The follow-up limit may have been reached.
- A repeated prompt may have been suppressed.

Scores differ from manual assessment

- Review the configured weights and input quality.
- Inspect each component score rather than relying only on the total.
- Use a larger recruiter-reviewed validation set before making production claims.

10. Data Protection and Human Review

Candidate resumes and interview answers can contain sensitive personal data:

- Use synthetic or approved test data during development.
- Do not log unnecessary candidate information.
- Restrict access to uploaded resumes and generated reports.

- Define retention and deletion behavior before production use.
- Apply appropriate authentication and transport security before exposing the API.
- Do not use automated scores as the sole basis for hiring decisions.

The current in-memory stores are suitable for demonstration/testing, not durable production storage.

11. Integration Limitations

The ATS API endpoints and HR Interview AI Python modules are documented separately because dedicated HR interview REST endpoints have not been confirmed. An API wrapper, persistent interview sessions, authentication, and production deployment controls require explicit implementation and tests.

12. Maintenance Checklist

- Update API documentation when routes or schemas change.
- Keep `data/api_contracts.json` aligned with actual route behavior.
- Update scoring documentation when configuration changes.
- Add regression tests for changed behavior.
- Run targeted and full test suites.
- Review the staged diff before committing.